

UNIVERSIDAD DEL VALLE DE GUATEMALA

Ingeniería de Software 1

Departamento de Ciencias de la computación

Sección 20

Erick Francisco Marroquín Rodríguez



Sistema de conexión estudiantil para intercambio de objetos y refuerzo académico.

GitHub: https://github.com/her24770/Proyecto-Software/tree/main/Corte_3/Base_Datos

Josué Hernández Gonzales – 24770

Jackelyn Nicolle Girón Villacinda – 24737

Alejandro Manuel Jerez Melgar – 24678

Juan Daniel Orodoñez Sierra – 24979

Oscar Rodrigo Rompich Cotzoyay – 24880

Sergio Estuardo Tan Coromac - 24759

Guatemala, 16 marzo 2026

Indice

Resumen	2
Introducción	3
Prototipos.....	4
Iteraciones:	5
Prototipo final:	5
Bitácora y evidencia de usuarios:	6
Lista de cambios entre cada iteración.	6
Cambios a requisitos funcionales	7
Análisis	8
Lista de requisitos funcionales.....	8
Diagrama de clases.....	13
Clases de tipo Entidad.....	14
Clases de tipo Controladora	19
Clases de tipo Interfaz.....	20
Diagrama de paquetes	22
Paquete: vista.....	22
Paquete: tiempo_real.....	23
Paquete: controlador	23
Paquete: autenticacion	24
Paquete: modelo	24
Paquete: persistencia	25
Paquete: repository.....	25
Paquete: infraestructura.....	25
Paquete: api_rest.....	26
Persistencia de datos	27
Diagrama de Entidad-Relación.....	27
Diagrama de Clases	27
Diseño.....	28
Arquitectura	28
Cliente Servidor	28

API REST.....	28
En Tiempo Real (WebSockets).....	29
Patrón de diseño	30
MVC (Model-View-Controller)	30
Repository.....	30
Singleton.....	31
Observer	31
Front-end	32
Next.js	32
TypeScript.....	32
CSS con metodología BEM	32
Socket.io Client	33
Zustand.....	33
React Hook Form	33
Date-fns.....	34
Back-end.....	34
Base de dato y almacenamiento	34
PostgreSQL	34
Redis	35
Cloudinary	36
Otros	36
Docker	36
Docker Compose.....	37
Git	37
GitHub	38
Planificación de tiempo	38
Planificación de tareas del equipo.....	39
LOGT.....	39
Conclusiones y reflexión de gestión de tiempo	42

Resumen

El presente trabajo consiste en el análisis, diseño y especificación técnica de un sistema digital de conexión estudiantil para la Universidad del Valle de Guatemala (UVG). La propuesta se centra en el desarrollo de una plataforma web que permita a los estudiantes intercambiar materiales académicos, vender o prestar objetos universitarios y ofrecer o solicitar tutorías dentro de la comunidad universitaria.

Actualmente, estas actividades se realizan de manera informal mediante contactos personales o grupos no estructurados, lo que genera desorganización, pérdida de tiempo, falta de visibilidad y problemas de confianza entre usuarios. Esta situación afecta especialmente a estudiantes de nuevo ingreso y a quienes no cuentan con una red amplia de contactos, evidenciando la necesidad de un canal centralizado, seguro y verificable que facilite estos intercambios académicos.

El objetivo del proyecto es definir y modelar un sistema estructurado que centralice estas interacciones, estableciendo requisitos funcionales y no funcionales medibles, diseñando la arquitectura del sistema, modelando sus componentes mediante diagramas UML y justificando técnicamente la selección de tecnologías a utilizar, garantizando seguridad, confiabilidad y escalabilidad.

Introducción

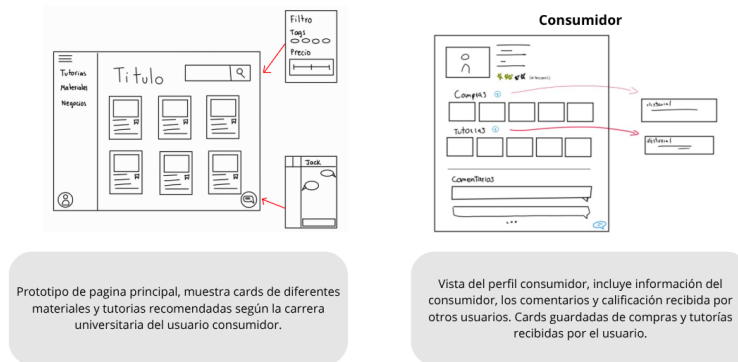
El presente proyecto está dirigido a la Universidad del Valle de Guatemala (UVG), una institución de educación superior reconocida por su excelencia académica. Atiende una comunidad compuesta por estudiantes, docentes y personal administrativo, los cuales participan activamente en procesos de enseñanza. A menudo se requieren recursos complementarios como lo son materiales de estudio, libros y apoyo académico. El proyecto se enfoca específicamente en el área de apoyo académico, con el objetivo de facilitar y optimizar el acceso a estos recursos.

La idea principal del proyecto consiste en el diseño de un sistema digital para la conexión estudiantil para facilitar la compra, venta y préstamo de materiales universitarios a su vez manejar de mejor manera la oferta y demanda de apoyo académico como lo son las tutorías entre estudiantes de UVG. La problemática de origen está en la informalidad que existe para realizar estas actividades, pues los estudiantes dependen de redes personales, grupos de mensajería y contactos directos, lo que genera desorganización, falta de confianza y dificultad para verificar la veracidad de los productos y servicios ofrecidos. Esta problemática no solo genera una pérdida de tiempo significativa las dos partes del acuerdo, sino que pone en desventaja a estudiantes de nuevo ingreso que no cuentan con una red amplia de contactos dentro de la universidad. Por lo tanto, surge la necesidad de crear una solución estructurada por medio de una plataforma digital, donde se permita la verificación institucional, ofrezca un entorno seguro y confiable para intercambios dentro de la universidad.

Prototipos

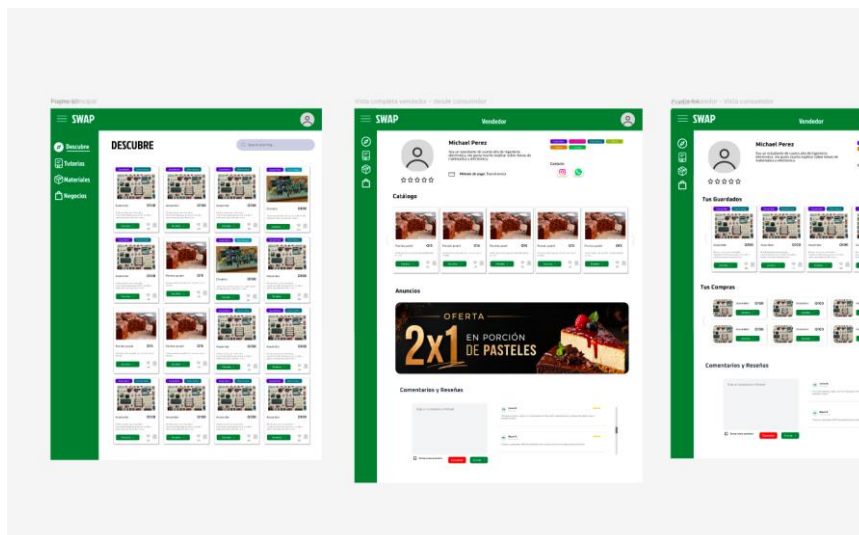
Se realizaron dos versiones de prototipos más centrándose en las historias de usuario más destacadas, modificándose entre cada versión.

Iteraciones:



En el siguiente [enlace](#) se puede visualizar los cambios en el prototipo de las fases y sus cambios. Una buena parte de los cambios fueron centrados en el desarrollo del apartado de moderación, agregando las vistas de moderación y reportes.

Prototipo final:



En el siguiente [enlace](#) se puede visualizar los cambios en el prototipo final del corte realizado en figma como una maquetación beta.

Bitácora y evidencia de usuarios:

En el siguiente [enlace](#) se puede visualizar los cambios en el prototipo final del corte realizado en figma como una maquetación beta. A todos los usuarios a los que se les

pidió retroalimentación coincidieron en que la solución ya se encuentra bastante completa, que están interesados en su uso y recomiendan implementar una página de presentación de la solución y una guía de uso.

Lista de cambios entre cada iteración.

Cuarta iteración con usuarios:

- Mostrar en el perfil de vendedor y tutor los métodos de pago que acepta el usuario.
- Agregar más opciones sobre incumplimientos en reporte.
- Subir imagen como evidencia en reporte.
- Horario semanal en perfil Tutor con sistema de colores para facilitar el proceso para agendar una tutoría.
- Agendar directamente desde el horario.

Quinta iteración con usuarios:

- Vista para agregar una publicación
- Vista general de los reportes, con información como el motivo, usuario y detalles
- Manejo de reportes para suspender o bloquear una cuenta

Iteración final

- Vista Inicio de sesión
- Vista registro
- Vista para editar perfil
- Se incluyen tags en card de tutorías y materiales
- Se incluye en card un botón para guardar y dar like
- Se agrega vista Descubre, incluye cards de todas las categorías
- Cards resumidas para mostrar compras del usuario
- Card para mostrar solo a tutor
- Se muestran tags relacionadas en perfiles vendedor, tutor y consumidor
- Se agrega botón a la vista para ver certificaciones
- Ventana emergente para ver y crear certificación
- Ventana emergente para crear nuevo producto
- Ventana emergente para crear nueva tutoría
- Ventana emergente para editar tutoría o producto
- Se agrega otros tipos de contacto en todos los perfiles
- Se agrega apartado de anuncios para perfil tutor
- Se agregan tres estados (Ocupado, Disponible, no disponible) a horario en perfil tutor

- Perfil vendedor visto desde un usuario vendedor
- Perfil tutor visto desde un usuario tutor
- Perfil consumidor visto desde usuario tutor o vendedor
- Perfil moderador
- Botón para ver perfil y editar perfil
- Comentarios con opción de enviar como anónimo
- Filtros se despliegan desde la barra de búsqueda
- Un usuario puede tener diferentes tipos de perfil (tutor, vendedor, consumidor)
- Vista de lista de usuarios, incluye estado, reportes y perfiles
- Se incluye diferentes tipos de reporte para publicación, mensaje o usuario
- Detalles sobre uno o varios reportes de un usuario
- Es posible suspender a un usuario por un tiempo determinado
- Ventana para pactar una entrega
- Cambio de color en acuerdos de entrega cuando ya se realizó o no se a realizado
- Silenciar o bloquear usuarios en mensajería

Cambios a requisitos funcionales

Tras los cambios e iteraciones hasta el prototipo final se encontraron el cambio d ciertas funcionalidades reflejados en la tabla de requisitos funcionales final. Los cambios para destacar son:

- Los reportes se manejarán por tipo o contexto. Como lo puede ser mensaje, publicación, usuario, etc.
- Deberá existir opción de bloquear o suspender por un tiempo algún usuario lo cual debe ser medible y establecido por base de datos.
- Cada persona puede tener más de un tipo de perfil e interactuar entre sus perfiles.
- El contacto con las personas debe ser opcional en el tema de brindar sus redes sociales personales.
- Los anuncios deben ser únicos por persona y deben ser visibles al momento de ver los detalles de los negocios o tutorías.
- Los perfiles tendrán diferentes perspectivas por lo que algunos perfiles se verán algunos componentes de una perspectiva y de otros perfiles no.
- Las etiquetas pueden referirse a otras etiquetas a la vez haciendo más fácil el manejo de categorías, así manejando áreas generales y los cursos a los cuales pertenecen a alguna categoría.
- La comunicación de usuarios por medio del chat debe acordarse más datos sobre cómo se llevará a cabo la entrega.
- La comunicación entre usuarios debe poder bloquearse para ambos en caso existe algun tipo de spam u otro motivo que lo amerite.

- Cada publicación debe ser diferente dependiendo del tipo del servicio, ya sea un material, producto de negocio o tutoría.
- Se deben de manejar de mejor forma la visibilidad de espacios disponibles de un tutor manejando tiempos ocupados, disponibles y no disponibles.

Análisis

Lista de requisitos funcionales

En la siguiente tabla se puede ver la lista de requisitos funcionales asociados a las historias de usuario.

#	Historia de usuario	Requisito funcional
1	Como estudiante de química, quiero un espacio para encontrar una bata blanca, para poder realizar laboratorios.	El sistema debe permitir filtrar por cursos o categorías relacionadas a una carrera específica.
2	Como estudiante de mecatrónica, quiero tutorías de Cálculo 1, para no perder la clase.	El sistema debe permitir encontrar a varios tutores que brindan un mismo curso. El sistema muestra la disponibilidad de los tutores
3	Como consumidor, quiero contactar a un tutor específico, para pactar una clase por medio de mensajes.	El sistema debe permitir buscar un perfil de tutor específico. El sistema debe mostrar el listado de todos los tutores según las categorías. El sistema debe permitir agendar una clase con un tutor y permite enviar mensajes al tutor después de agendar.
4	Como tutor, quiero estar de acuerdo con un estudiante, para organizar mi tiempo para dar la tutoría.	El sistema debe mostrar la disponibilidad del tutor por medio de un calendario semanal. El sistema debe permitir que un estudiante contacte al tutor.
5	Como vendedor, quiero contestar mensajes en orden pues me escribieron para comprar de mi emprendimiento, para darles prioridad a los que escribieron primero.	El sistema debe mostrar la hora de contacto, al enviar un mensaje. El sistema debe priorizar los mensajes más antiguos.
6	Como vendedor, quiero poder enviar más fotos de mi emprendimiento, para crear más confianza con mi cliente y vender más.	- El sistema debe permitir subir múltiples imágenes a una publicación. -El sistema deberá permitir ver todas las imágenes de un producto. -El sistema deberá permitir eliminar o reemplazar imágenes de una publicación.

7	Como tutor, tuve problemas con un estudiante en una clase, quiero poder ver el historial de mensajes, para poder respaldar si se da un problema en el futuro.	- El sistema deberá almacenar el historial de mensajes entre los usuarios. -El sistema debe permitirles a los usuarios consultar mensajes anteriores. -El sistema deberá mostrar fecha de cada mensaje con el usuario.
8	Como comprador que encontró lo que buscaba, quiero ver el perfil del vendedor y los artículos publicados para saber si tiene otra cosa que me interese.	-El sistema debe permitir que los usuarios accedan al perfil de otros usuarios. -El sistema deberá mostrar la información del usuario en su perfil. -el sistema debe mostrar la lista de productos publicados por el vendedor. -El sistema debe permitir ver los detalles de cada artículo publicado.
9	Como vendedor con varios artículos, quiero tener un espacio donde se agrupen todos mis productos para administrarlos fácilmente.	-El sistema debe permitir a los vendedores ver todos sus productos en un solo espacio. -El sistema debe permitir al vendedor editar la información de cada uno de sus productos publicados. -El sistema le debe permitir al vendedor administrar todos sus productos desde un mismo panel.
10	Como vendedor, quiero cambiar el estado de mis artículos (disponible, vendido o reservado) para mostrar su disponibilidad real.	-El sistema le debe permitir a los vendedores modificar el estado de sus productos. -El sistema debe mostrar el estado del producto. -El sistema debe actualizar el estado del producto luego de ser modificado.
11	Como comprador que adquirió un artículo o servicio, quiero calificar y dejar una reseña al vendedor o tutor para compartir mi experiencia.	- El sistema debe permitir al comprador calificar al vendedor/tutor con una escala, de 1 a 5 estrellas. - El sistema debe permitir al comprador escribir una reseña en texto libre. - El sistema debe validar que solo los compradores que hayan adquirido el producto o servicio puedan dejar una reseña. - El sistema debe asociar la reseña al vendedor/tutor correspondiente. - El sistema debe mostrar la fecha en que se realizó la reseña. - El sistema debe permitir al comprador editar o eliminar su reseña (según reglas del negocio). - El sistema debe actualizar automáticamente la calificación promedio del vendedor/tutor al agregarse una nueva reseña.
12	Como nuevo vendedor, quiero agregar información sobre mí y mis datos de contacto para que los clientes conozcan lo que ofrezco.	- El sistema debe permitir al vendedor crear y editar su perfil. - El sistema debe permitir ingresar información personal (nombre, descripción, experiencia, etc.). - El sistema debe permitir agregar datos de contacto, otros medios definidos por la plataforma. - El sistema debe validar los campos obligatorios antes de guardar la información. - El sistema debe permitir actualizar la información del perfil en cualquier momento. - El sistema debe mostrar esta información en una sección pública del perfil del vendedor.

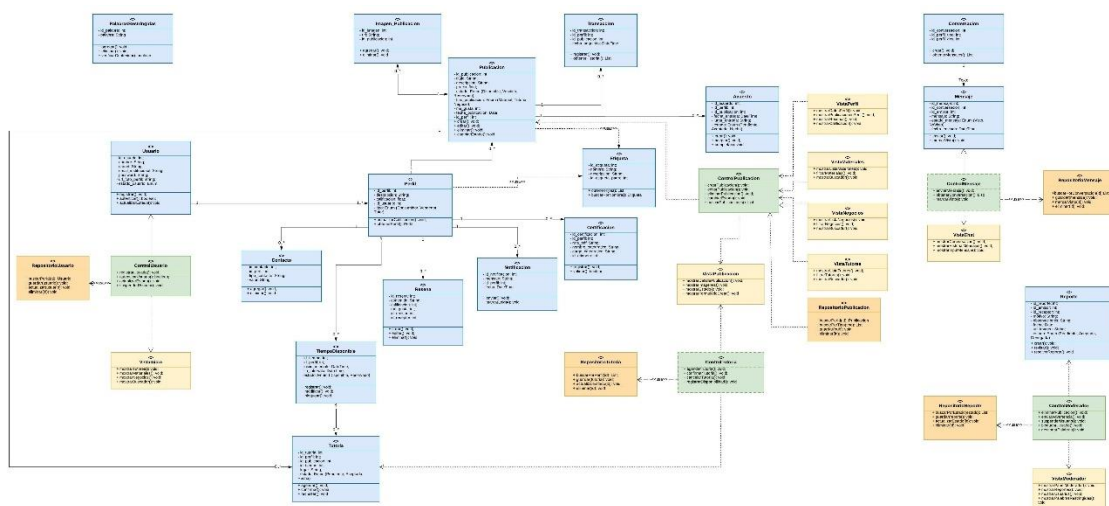
13	Como vendedor, quiero agregar características y detalles a mis productos para que puedan encontrarlos fácilmente en el buscador.	El sistema debe permitir al vendedor crear nuevos productos o servicios. El sistema debe permitir agregar título, descripción y categoría al producto. El sistema debe permitir agregar características específicas (precio, disponibilidad, estado, modalidad, etc.). El sistema debe permitir agregar palabras clave (tags) para mejorar la búsqueda. El sistema debe validar que los campos obligatorios estén completos antes de publicar el producto. El sistema debe indexar los productos para que puedan ser encontrados en el buscador. El sistema debe permitir editar o eliminar productos publicados.
14	Como vendedor, quiero visualizar mis reseñas y calificación general para generar confianza en futuros clientes.	El sistema debe mostrar al vendedor una lista de todas las reseñas recibidas. El sistema debe mostrar la calificación promedio general. El sistema debe mostrar el número total de reseñas recibidas. El sistema debe permitir filtrar reseñas por fecha o calificación. El sistema debe mostrar detalles de cada reseña (comentario, puntuación, fecha). El sistema debe actualizar automáticamente la información cuando se agregue una nueva reseña.
15	Como estudiante que busca una tutoría, quiero conocer más información sobre el tutor para decidir si es adecuado para mí.	El sistema debe permitir al estudiante acceder al perfil público del tutor. El sistema debe mostrar información personal y profesional del tutor. El sistema debe mostrar las materias que el tutor imparte. El sistema debe mostrar reseñas y calificación promedio del tutor. El sistema debe permitir visualizar disponibilidad u horarios del tutor.
16	Como tutor, quiero indicar las materias en las que puedo dar tutorías para que los estudiantes puedan encontrarme al buscar por curso.	El sistema debe permitir al tutor seleccionar o registrar las materias que imparte. El sistema debe permitir actualizar o eliminar materias registradas. El sistema debe vincular las materias al perfil público del tutor. El sistema debe indexar las materias para que puedan encontrarse mediante el buscador por curso o palabra clave.
17	Como tutor que imparte varias materias, quiero mostrar todas las clases en las que estoy capacitado para que los estudiantes evalúen si puedo ayudarlos.	El sistema deberá permitir al tutor registrar los cursos que puede impartir. El sistema deberá permitir al tutor editar y eliminar cursos registradas. El sistema deberá mostrar en el perfil del tutor la lista completa de cursos que imparte. El sistema deberá permitir asociar cada curso a sus etiquetas de filtros.
18	Como tutor, quiero mostrar mi disponibilidad de horario para	El sistema deberá permitir al tutor registrar su disponibilidad por día y hora.

	facilitar la organización de tutorías.	El sistema deberá permitir modificar o actualizar la disponibilidad. El sistema deberá mostrar la disponibilidad del tutor en su perfil. El sistema deberá bloquear horarios marcados como no disponibles o ya ocupados.
19	Como estudiante interesado en un tutor, quiero ver su disponibilidad para saber si su horario se adapta al mío.	El sistema deberá permitir al estudiante visualizar la disponibilidad del tutor. El sistema deberá mostrar los horarios disponibles en un formato claro (tabla o calendario). El sistema deberá actualizar automáticamente la disponibilidad cuando un horario sea reservado.
20	Como estudiante que quiere agendar una tutoría, quiero poder contactar rápidamente al tutor para coordinar la sesión.	El sistema deberá permitir al estudiante enviar un mensaje directo al tutor. El sistema deberá permitir iniciar un chat desde el perfil del tutor. El sistema deberá notificar al tutor cuando reciba una solicitud de tutoría. El sistema deberá permitir confirmar o rechazar una solicitud de tutoría. El sistema deberá registrar el historial de conversaciones.
21	Como tutor, quiero ver el feedback y reseñas de mis estudiantes para mejorar mi servicio.	El sistema deberá permitir al estudiante dejar una reseña después de una tutoría. El sistema deberá permitir asignar una calificación numérica al tutor. El sistema deberá mostrar al tutor su calificación promedio. El sistema deberá permitir al tutor visualizar todas las reseñas recibidas. El sistema deberá asociar cada reseña a una tutoría específica.
22	Como moderador, quiero visualizar todas las publicaciones realizadas para identificar posibles incumplimientos de las normas de compra y venta.	El sistema deberá permitir al moderador visualizar todas las publicaciones activas. El sistema deberá permitir al moderador acceder al detalle completo de cada publicación. El sistema deberá permitir al moderador eliminar publicaciones que incumplan normas. El sistema deberá registrar la acción de eliminación junto con una justificación. El sistema deberá permitir suspender temporalmente a un usuario. El sistema deberá mantener un registro histórico de sanciones.
23	Como moderador, quiero eliminar publicaciones que incumplan las normas, dejando una justificación visible, para mantener un ambiente seguro dentro de la plataforma.	El sistema debe permitir a los moderadores eliminar publicaciones o comentarios que incumplan las normas. El sistema debe solicitar al moderador ingresar una justificación al eliminar una publicación. El sistema debe mostrar la justificación de la eliminación al autor de la publicación. El sistema debe registrar en un historial las publicaciones eliminadas y la razón de su eliminación. El sistema debe permitir buscar y filtrar publicaciones reportadas o sospechosas de incumplir normas.

		El sistema debe notificar al usuario cuando una de sus publicaciones ha sido eliminada.
24	Como moderador, quiero enviar advertencias a usuarios que cometan faltas leves para fomentar el cumplimiento de las normas.	El sistema debe permitir a los moderadores enviar advertencias a usuarios registrados. El sistema debe permitir seleccionar el tipo de advertencia (por ejemplo: leve, moderada, grave). El sistema debe mostrar al usuario la advertencia recibida junto con su motivo. El sistema debe almacenar el historial de advertencias de cada usuario. El sistema debe permitir buscar usuarios por nombre, correo o ID para emitir advertencias.
25	Como moderador, quiero mantener una lista de palabras inadecuadas para facilitar la detección de publicaciones o comentarios que incumplan las normas.	El sistema debe permitir a los moderadores crear, modificar y eliminar palabras o frases prohibidas. El sistema debe analizar automáticamente publicaciones y comentarios en busca de palabras prohibidas. El sistema debe marcar o reportar automáticamente contenido que contenga palabras restringidas. El sistema debe permitir actualizar las normas y reglas de la plataforma. El sistema debe mostrar las normas vigentes a los usuarios dentro de la plataforma.
26	Como moderador, quiero suspender o bloquear a usuarios que incumplan el reglamento para mantener un entorno seguro dentro de la plataforma.	El sistema debe permitir a los moderadores suspender temporalmente cuentas de usuario. El sistema debe permitir a los moderadores bloquear permanentemente cuentas de usuario. El sistema debe registrar el motivo y duración de la suspensión. El sistema debe notificar al usuario cuando su cuenta sea suspendida o bloqueada. El sistema debe impedir el acceso a la plataforma a los usuarios bloqueados.
27	Como estudiante, quiero un espacio para poder sacarle provecho a los materiales que ya no utilizo.	El sistema debe permitir a los estudiantes crear publicaciones para vender o alquilar materiales académicos. El sistema debe permitir agregar información del material (nombre, curso, estado, precio, imágenes). El sistema debe permitir editar o eliminar publicaciones propias. El sistema debe permitir visualizar publicaciones de materiales académicos disponibles.
28	Como estudiante, quiero reducir la inversión en materiales para los cursos que lo requieren y tener alternativas para conseguirlos.	El sistema debe permitir visualizar el precio de los materiales publicados. El sistema debe permitir indicar si el material está disponible para venta o alquiler. El sistema debe permitir buscar o filtrar materiales por curso, tipo o precio.
29	Como estudiante quiero compartir los materiales académicos que tengo para ayudar a estudiantes que lo requieran.	El sistema permitirá a los estudiantes crear publicaciones para compartir materiales académicos. El sistema podrá almacenar y asociar los materiales publicados con el vendedor que los publicó.

30	Como moderador quiero que los estudiantes tengan claro que materiales pueden promocionar para reducirme la carga de trabajo.	El sistema deberá mostrar los acuerdos de uso y una guía de materiales permitidos dentro de la plataforma. El sistema deberá solicitar la aceptación obligatoria de los términos y condiciones antes de permitir el uso de la aplicación.
31	Como consumidor, quiero ver si sigue disponible el producto que está en venta para que no haya confusiones con el vendedor.	El sistema deberá mostrar el estado de disponibilidad de cada producto (disponible, reservado o vendido) dentro de su publicación.
32	Como consumidor, quiero ver mi calificación para ver cómo me evalúan los demás usuarios.	El sistema deberá mostrar la calificación que otros usuarios han otorgado al perfil de un usuario dentro de la plataforma. El sistema deberá permitir a un usuario visualizar la calificación promedio que ha recibido de otros usuarios.
33	Como consumidor, quiero ver el historial de los productos que he comprado para tener bien claro que he comprado en el sitio web	El sistema deberá mostrar un historial de los productos comprados o intercambiados por el usuario, incluyendo información básica del producto y del vendedor.
34	Como consumidor quiero ver el historial de las tutorías que he tomado para llevar control de los tutores con los que me he comunicado.	El sistema deberá mostrar un historial detallado de las tutorías recibidas por el estudiante, incluyendo tutor, materia y fecha de la sesión.

Diagrama de clases



En el siguiente [enlace](#) se encuentra a detalle el Diagrama de clases. El diagrama presenta la arquitectura del sistema, incluyendo a las entidades de usuarios, publicaciones, tutorías y mensajes, incluyendo las relaciones entre ellas. Organiza la lógica en las capas de modelo, controladores, vistas y repositorios, siguiendo el patrón MVC.

Clases de tipo Entidad

USUARIO

- **Estereotipo:** Entidad
- **Descripción:** Representa a cualquier persona registrada en la plataforma. Almacena credenciales de acceso y estado de la cuenta. Es la clase raíz de la que dependen todos los perfiles.
- **Atributos principales:** id_usuario, nombre, carnet, email_institucional, password, url_foto_perfil, estado_usuario (Activo/Suspendido/Bloqueado).
- **Métodos:** registrar(), autenticar(), actualizarEstado().
- **Requisitos cubiertos:** RF de registro e inicio de sesión, suspensión y bloqueo de cuentas (HU 22, 26).

PERFIL

- **Estereotipo:** Entidad
- **Descripción:** Un usuario puede tener hasta tres tipos de perfil: Consumidor, Vendedor y Tutor. Cada perfil tiene su propia calificación y descripción. La distinción por tipo permite aplicar reglas de negocio específicas según el rol.
- **Atributos principales:** id_perfil, descripcion, calificacion, id_usuario, tipo (Consumidor/Vendedor/Tutor).
- **Métodos:** actualizarCalificacion(), obtenerPerfil().
- **Requisitos cubiertos:** Gestión de perfil (HU 8, 12, 14, 15, 32).

PUBLICACION

- **Estereotipo:** Entidad
- **Descripción:** Núcleo del sistema de intercambio. Representa cualquier artículo, servicio de tutoría o negocio publicado por un perfil vendedor o tutor. Diferencia el tipo de publicación para aplicar lógica específica (ej.: tutoría tiene horario, material tiene estado físico).
- **Atributos principales:** id_publicacion, titulo, descripción, precio, estado (Disponible/Vendido/Reservado), tipo_publicacion (Material/Tutoría/Negocio), me_gusta, fecha_publicacion, id_perfil.
- **Métodos:** crear(), editar(), eliminar(), cambiarEstado().
- **Requisitos cubiertos:** Publicación de materiales y tutoría (HU 6, 9, 10, 13, 27, 28, 29, 31).

ETIQUETA

- **Estereotipo:** Entidad

- **Descripción:** Sistema jerárquico de categorías para clasificar publicaciones y perfiles. Permite filtros por carrera, curso o tipo de artículo. Una etiqueta puede tener una etiqueta padre, formando una taxonomía (ej.: Ciencias -> Química -> Laboratorio).
- **Atributos principales:** id_etiqueta, nombre, descripcion, id_etiqueta_padre.
- **Métodos:** obtenerHijos(), buscarPorNombre().
- **Requisitos cubiertos:** Filtrado por categorías y búsqueda (HU 1, 13, 16, 17).

IMAGEN_PUBLICACION

- **Estereotipo:** Entidad
- **Descripción:** Almacena las imágenes asociadas a una publicación. Al permitir múltiples imágenes por publicación, el vendedor puede mostrar diferentes ángulos o detalles del producto para generar confianza con el comprador.
- **Atributos principales:** id_imagen, url, id_publicacion.
- **Métodos:** agregar(), eliminar().
- **Requisitos cubiertos:** Subir múltiples imágenes (HU 6).

RESEÑA

- **Estereotipo:** Entidad
- **Descripción:** Registra la evaluación que un perfil hace sobre otro tras completar una transacción o tutoría. Incluye calificación numérica y comentario textual. Las reseñas son bidireccionales: un emisor evalúa a un receptor.
- **Atributos principales:** id_resena, contenido, calificacion, me_gusta, id_emisor, id_receptor.
- **Métodos:** crear(), editar(), eliminar().
- **Requisitos cubiertos:** Sistema de calificaciones y reseñas (HU 11, 14, 21, 32).

CONVERSACION

- **Estereotipo:** Entidad
- **Descripción:** Representa el canal de comunicación privado entre exactamente dos perfiles. Agrupa todos los mensajes intercambiados entre ellos. Una conversación se crea cuando un usuario contacta a otro por primera vez.
- **Atributos principales:** id_conversacion, id_perfil_uno, id_perfil_dos.
- **Métodos:** crear(), obtenerMensajes().
- **Requisitos cubiertos:** Chat entre usuarios (HU 3, 4, 5, 7, 20).

MENSAJE

- **Estereotipo:** Entidad

- **Descripción:** Unidad atómica de comunicación dentro de una conversación. Registra el contenido, el emisor, la fecha y el estado de lectura. Permite reconstruir el historial completo de intercambios entre dos usuarios.
- **Atributos principales:** id_mensaje, id_conversacion, id_emisor, mensaje, estado_mensaje (Visto/NoVisto), fecha_enviado.
- **Métodos:** enviar(), marcarVisto().
- **Requisitos cubiertos:** Historial de mensajes, prioridad por orden de llegada (HU 5, 7, 20).

TIEMPO DISPONIBLE

- **Estereotipo:** Entidad
- **Descripción:** Define los intervalos de tiempo en que un tutor está disponible para dar sesiones. El estado del intervalo cambia automáticamente a Reservado cuando un estudiante agenda una tutoría en ese horario, evitando dobles reservaciones.
- **Atributos principales:** id_tiempo, id_perfil, inicio_intervalo, fin_intervalo, estado (Disponible/Reservado).
- **Métodos:** registrar(), modificar(), bloquear().
- **Requisitos cubiertos:** Disponibilidad del tutor (HU 2, 4, 18, 19).

TUTORIA

- **Estereotipo:** Entidad
- **Descripción:** Registra el acuerdo formal entre un estudiante y un tutor para una sesión específica. Vincula la publicación de tutoría, el intervalo de tiempo reservado y el lugar de la sesión. Su estado evoluciona desde Pendiente hasta Hecha.
- **Atributos principales:** id_tutoria, id_perfil, id_publicacion, id_tiempo, lugar, estado (Pendiente/Aceptada/Hecha).
- **Métodos:** agendar(), confirmar(), rechazar().
- **Requisitos cubiertos:** Agendamiento de tutorías (HU 3, 4, 20, 34).

ACUERDO

- **Estereotipo:** Entidad
- **Descripción:** Formaliza el pacto entre un comprador y un vendedor para la entrega de un artículo físico. Registra el lugar y fecha acordados para el intercambio. Provee trazabilidad de la transacción sin manejar pagos electrónicos.
- **Atributos principales:** id_acuerdo, id_perfil, id_publicacion, fecha_entrega, lugar_entrega, estado (Pendiente/Aceptada/Hecha).

- **Métodos:** crear(), aceptar(), completar().
- **Requisitos cubiertos:** Coordinación de entrega de materiales (HU 3, 9, 33).

TRANSACCION

- **Estereotipo:** Entidad
- **Descripción:** Historial inmutable de todas las adquisiciones realizadas por un consumidor, incluyendo materiales comprados o prestados y tutorías tomadas. Permite al usuario revisar su historial de actividad en la plataforma.
- **Atributos principales:** id_transaccion, id_perfil, id_publicacion, fecha_adquirido.
- **Métodos:** registrar(), obtenerHistorial().
- **Requisitos cubiertos:** Historial de compras y tutorías (HU 33, 34).

REPORTE

- **Estereotipo:** Entidad
- **Descripción:** Mecanismo de denuncia entre usuarios. Un perfil puede reportar a otro adjuntando evidencia visual y una descripción del incidente. El moderador revisa el reporte y decide aceptarlo o denegarlo, actualizando el estado correspondiente.
- **Atributos principales:** id_reporte, id_emisor, id_receptor, motivo, observaciones, fecha, url_imagen, estado (Pendiente/Aceptada/Denegada).
- **Métodos:** crear(), revisar(), resolverReporte().
- **Requisitos cubiertos:** gestión de reportes por moderadores (HU 22, 23).

NOTIFICACION

- **Estereotipo:** Entidad
- **Descripción:** Mensajes del sistema dirigidos a un perfil específico para informarle sobre eventos relevantes como recepción de mensajes, confirmación de tutorías, cambios en publicaciones o acciones del moderador sobre su cuenta.
- **Atributos principales:** id_notificacion, mensaje, id_perfil, fecha.
- **Métodos:** enviar(), marcarLeida().
- **Requisitos cubiertos:** Notificaciones del sistema (HU 20, 23, 26).

PALABRASRESTRINGIDAS

- **Estereotipo:** Entidad
- **Descripción:** Glosario de términos inadecuados mantenido por los moderadores. El sistema analiza automáticamente el contenido de

publicaciones y comentarios contra esta lista. Permite mantener un ambiente respetuoso sin requerir revisión manual de todo el contenido.

- **Atributos principales:** id_palabra, palabra.
- **Métodos:** agregar(), eliminar(), verificarContenido().
- **Requisitos cubiertos:** Filtrado automático de contenido (HU 25).

CERTIFICACION

- **Estereotipo:** Entidad
- **Descripción:** Avala la competencia de un tutor en una materia específica mediante comprobante firmado por un catedrático de la UVG. Vincula la certificación a una etiqueta de curso, de modo que aparezca en búsquedas de esa materia y genere confianza en el estudiante.
- **Atributos principales:** id_certificacion, id_perfil, ruta_pdf, nombre_catedratico, cargo_catedratico, id_etiqueta.
- **Métodos:** registrar(), validar().
- **Requisitos cubiertos:** Credenciales del tutor (HU 15, 16).

CONTACTO

- **Estereotipo:** Entidad
- **Descripción:** Información de contacto adicional asociada a un perfil, como número de teléfono u otros medios de comunicación externos a la plataforma. Un perfil puede tener múltiples datos de contacto de diferentes tipos.
- **Atributos principales:** id_contacto, id_perfil, tipo_contacto, valor.
- **Métodos:** agregar(), eliminar().
- **Requisitos cubiertos:** Datos de contacto del vendedor/tutor (HU 12).

Clases de tipo Controladora

ControlUsuario

- **Estereotipo:** Controladora
- **Descripción:** Gestiona todo el ciclo de vida de un usuario: registro, autenticación, actualización de estado y acciones administrativas como suspensión o bloqueo. Valida credenciales y aplica reglas de acceso.
- **Métodos principales:** registrarUsuario(), autenticarUsuario(), actualizarEstado(), suspenderUsuario().

ControlPublicacion

- **Estereotipo:** Controladora

- **Descripción:** Centraliza toda la lógica relacionada con publicaciones: creación, edición, eliminación, cambio de estado y búsqueda con filtros. Coordina la carga de imágenes y la asociación de etiquetas.
- **Métodos principales:** crearPublicacion(), editarPublicacion(), eliminarPublicacion(), cambiarEstado(), buscarPublicaciones().

ControlTutoria

- **Estereotipo:** Controladora
- **Descripción:** Maneja el flujo completo de tutoría: desde el registro de disponibilidad del tutor hasta la confirmación o rechazo de una solicitud de sesión. Bloquea los horarios reservados y registra el estado de cada tutoría.
- **Métodos principales:** agendarTutoria(), confirmarTutoria(), cancelarTutoria(), registrarDisponibilidad().

ControlMensaje

- **Estereotipo:** Controladora
- **Descripción:** Administra las conversaciones y mensajes entre usuarios. Crea nuevas conversaciones, enviar mensajes dentro de ellas, marca mensajes como leídos y recupera el historial ordenado cronológicamente.
- **Métodos principales:** enviarMensaje(), obtenerConversacion(), marcarVisto().

ControlModerador

- **Estereotipo:** Controladora
- **Descripción:** Provee todas las herramientas de administración y moderación: eliminar publicaciones con justificación, enviar advertencias, suspender o bloquear usuarios y gestionar el glosario de palabras restringidas.
- **Métodos principales:** eliminarPublicacion(), enviarAdvertencia(), suspenderUsuario(), bloquearUsuario(), gestionarPalabras().

Clases de tipo Interfaz

VistaInicio

- **Estereotipo:** Interfaz
- **Descripción:** Página principal de la plataforma. Muestra simultáneamente tutores, materiales y negocios disponibles, con buscador general. Sirve como punto de entrada al sistema y permite al usuario explorar todas las categorías antes de ir a una vista especializada.
- **Métodos:** mostrarTutores(), mostrarMateriales(), mostrarNegocios(), mostrarBuscador().
- **Conectada a:** ControlPublicacion.

VistaTutores

- **Estereotipo:** Interfaz
- **Descripción:** Pagina dedicada exclusivamente a mostrar perfiles de tutores disponibles. Permite filtrar por materia, calificación o disponibilidad horaria. El usuario puede ver el detalle de cada tutor y contactarlo.
- **Métodos:** mostrarListaTutores(), filtrarTutores(), mostrarBuscador().
- **Conectada a:** ControlPublicacion.

VistaMateriales

- **Estereotipo:** Interfaz
- **Descripción:** Pagina dedicada exclusivamente a mostrar publicaciones de materiales académicos en venta o préstamo. Permite filtrar por curso, estado del artículo o precio.
- **Métodos:** mostrarListaMateriales(), filtrarMateriales(), mostrarBuscador().
- **Conectada a:** ControlPublicacion.

VistaNegocios

- **Estereotipo:** Interfaz
- **Descripción:** Pagina dedicada exclusivamente a mostrar publicaciones de negocios estudiantiles. Permite explorar productos y servicios ofrecidos por la comunidad universitaria y filtrar por categoría o precio.
- **Métodos:** mostrarListaNegocios(), filtrarNegocios(), mostrarBuscador().
- **Conectada a:** ControlPublicacion.

VistaPerfil

- **Estereotipo:** Interfaz
- **Descripción:** Vista pública del perfil de cualquier usuario, ya sea vendedor o tutor. Muestra información personal, publicaciones activas, calificación promedio y reseñas recibidas. También sirve como perfil propio para edición de datos.
- **Metodos:** mostrarDatosPerfil(), mostrarPublicacionesPerfil(), mostrarResenas(), mostrarCalificacion().
- **Conectada a:** ControlUsuario, ControlPublicacion.

VistaPublicacion

- **Estereotipo:** Interfaz
- **Descripción:** Muestra el detalle completo de una publicación, incluyendo título, descripción, imágenes, precio, estado y datos del vendedor o tutor. También provee el formulario para crear o editar publicaciones.

- **Métodos:** mostrarDetallePublicacion(), mostrarImagenes(), mostrarEstado(), mostrarFormularioCrear().
- **Conectada a:** ControlPublicacion, ControlTutoria.

VistaChat

- **Estereotipo:** Interfaz
- **Descripción:** Interfaz de mensajería que muestra el historial de mensajes de una conversación, ordenados cronológicamente, y permite al usuario escribir y enviar nuevos mensajes.
- **Métodos:** mostrarConversacion(), mostrarHistorialMensajes(), mostrarInputMensaje().
- **Conectada a:** ControlMensaje.

VistaModerador

- **Estereotipo:** Interfaz
- **Descripción:** Panel de administración exclusivo para el usuario administrador. Permite revisar y resolver reportes generados por usuarios, gestionar cuentas (suspender, bloquear), ver el listado completo de usuarios y administrar el glosario de palabras restringidas.
- **Métodos:** mostrarPanelModerador(), mostrarReportes(), mostrarUsuarios(), mostrarPalabrasRestringidas().
- **Conectada a:** ControlModerador

Clases de tipo Repository

RepositorioUsuario

Estereotipo: Repository

Descripción: Encapsula todas las operaciones de base de datos relacionadas con Usuario y Perfil. El ControlUsuario llama sus métodos sin conocer el esquema de Prisma subyacente.

Métodos: buscarPorId(id), guardar(usuario), actualizar(usuario), eliminar(id).

Conectado a: ControlUsuario.

RepositorioPublicacion

Estereotipo: Repository

Descripción: Encapsula las operaciones sobre Publicacion, Imagen_Publicacion, Etiqueta y sus tablas de union. Provee búsquedas filtradas por tipo (Material, Tutoria, Negocio) que ControlPublicacion utiliza para alimentar las cuatro vistas de listado.

Métodos: buscarPorId(id), buscarPorTipo(tipo), guardar(pub), eliminar(id).

Conectado a: ControlPublicacion.

RepositorioMensaje

Estereotipo: Repository

Descripción: Encapsula las operaciones sobre Conversacion y Mensaje. Permite recuperar el historial completo de una conversación ordenado cronologicamente y persistir nuevos mensajes enviados via Socket.io.

Métodos: buscarPorConversacion(id), guardar(mensaje), marcarVisto(id), eliminar(id).

Conectado a: ControlMensaje.

RepositorioTutoria

Estereotipo: Repository

Descripción: Encapsula las operaciones sobre TiempoDisponible, Tutoria y Acuerdo. Centraliza la lógica de consulta de disponibilidad de tutores y el cambio de estado de sesiones agendadas.

Métodos: buscarPorPerfil(id), guardar(tutoria), actualizarEstado(id), eliminar(id).

Conectado a: ControlTutoria.

RepositorioReporte

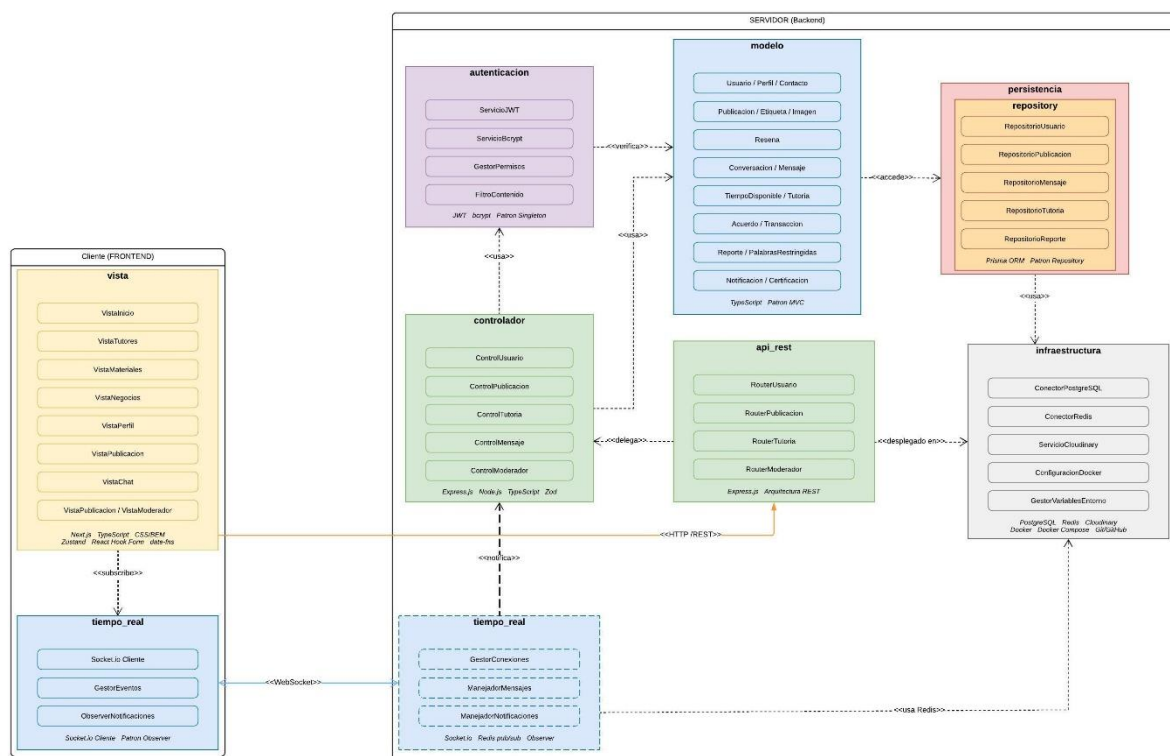
Estereotipo: Repository

Descripción: Encapsula las operaciones sobre Reporte, Transaccion y PalabrasRestringidas. Permite al moderador consultar reportes pendientes y actualizar su estado tras revision.

Métodos: buscarPorEstado(estado), guardar(reporte), actualizarEstado(id), eliminar(id).

Conectado a: ControlModerador.

Diagrama de paquetes



En el siguiente [enlace](#) se encuentra a detalle el Diagrama de clases. El diagrama representa la arquitectura bajo el enfoque Cliente-Servidor, separando las responsabilidades en capas como vista, controlador, modelo, persistencia e infraestructura. Integra patrones de diseño y tecnologías como API REST y Websockets para comunicación en tiempo real.

Paquete: vista

Responsabilidad: Capa de presentación del sistema. Contiene todas las pantallas con las que el usuario interactúa directamente. No contiene lógica de negocio; obtiene datos del controlador vía API REST y recibe eventos en tiempo real vía Socket.io.

Tecnologías: Next.js, TypeScript, CSS puro con metodología BEM, Zustand (estado global), React Hook Form (formularios), date-fns (fechas), Socket.io Client.

Patrón: Vista del MVC. Zustand implementa el patrón Observer para sincronizar estado entre componentes.

Acoplamiento: Depende de: controlador (via API REST), tiempo_real (via WebSocket). No accede a modelo ni persistencia directamente.

Componentes:

- VistaInicio: Muestra tutores, materiales y negocios juntos en una sola página.
- VistaTutores: Pagina exclusiva para explorar y filtrar tutores disponibles.
- VistaMateriales: Pagina exclusiva para explorar y filtrar materiales académicos.

- VistaNegocios: Pagina exclusiva para explorar y filtrar negocios estudiantiles.
- VistaPerfil: Visualización de perfiles de vendedores, tutores y perfil propio.
- VistaPublicacion: Detalle y formulario de creación/edición de publicaciones.
- VistaChat: Interfaz de mensajería en tiempo real entre usuarios.
- VistaModerador: Panel de administración exclusivo para el usuario administrador.

Paquete: tiempo_real

Responsabilidad: Gestiona la comunicación bidireccional en tiempo real entre el servidor y los clientes conectados. Maneja la entrega instantánea de mensajes de chat y notificaciones sin necesidad de recargar la página.

Tecnologías: Socket.io (servidor y cliente), Redis (pub/sub para escalar conexiones entre múltiples instancias del servidor).

Patrón: Observer: los clientes se suscriben a canales (rooms de Socket.io) y reciben eventos cuando ocurre un cambio relevante. Redis actúa como broker de mensajes entre instancias.

Arquitectura: Tiempo Real con WebSockets sobre la arquitectura Cliente-Servidor.

Acoplamiento: Depende de: controlador (para persistir mensajes), infraestructura (Redis). Es suscrito por: vista.

Componentes:

- GestorConexiones: Administra la apertura y cierre de conexiones WebSocket por usuario.
- ManejadorMensajes: Recibe, valida y distribuye mensajes de chat en tiempo real.
- ManejadorNotificaciones: Emite eventos de notificación a perfiles específicos.

Paquete: controlador

Responsabilidad: Capa de lógica del negocio del backend. Orquesta las operaciones del sistema: valida entradas con Zod, aplica reglas de negocio, coordina las entidades y devuelve respuestas HTTP al cliente.

Tecnologías: Node.js, Express.js, TypeScript, Zod (validacion de schemas).

Patron: Controlador del MVC. Recibe peticiones HTTP desde api_rest, aplica logica y delega en modelo.

Acoplamiento: Depende de: modelo, autenticación, api_rest. Recibe solicitudes de: vista (via api_rest), tiempo_real.

Componentes:

- ControlUsuario: Registro, autenticación y gestión del ciclo de vida de cuentas.
- ControlPublicacion: Creación, edición, eliminación y búsqueda de publicaciones.

- ControlTutoria: Agenda, disponibilidad y confirmación de sesiones de tutoría.
- ControlMensaje: Gestión de conversaciones y persistencia de mensajes.
- ControlModerador: Herramientas de moderación, reportes y gestión de usuarios.

Paquete: autenticacion

Responsabilidad: Servicio transversal de seguridad. Gestiona la identidad del usuario: emite y verifica tokens JWT, cifra contraseñas con bcrypt, controla permisos por rol y filtra contenido inapropiado antes de persistirlo.

Tecnologías: JSON Web Token (JWT), bcrypt.

Patron: Singleton: las instancias de ServicioJWT y ServicioBcrypt se instancian una sola vez y se reutilizan en todo el sistema. Middleware de Express para interceptar peticiones.

Acoplamiento: Depende de: modelo (para verificar estado del usuario). Es usada por: controlador en cada petición protegida.

Componentes:

- ServicioJWT: Genera tokens de acceso al login y los verifica en cada petición protegida.
- ServicioBcrypt: Cifra la contraseña al registrar y la compara al autenticar.
- GestorPermisos: Middleware que valida que el rol del usuario (consumidor, vendedor, tutor, admin) le permita ejecutar la acción solicitada.
- FiltroContenido: Analiza el texto de publicaciones y comentarios contra PalabrasRestringidas antes de guardarlos.

Paquete: modelo

Responsabilidad: Capa de dominio. Define las entidades del sistema con sus atributos y comportamientos. Representa el estado del negocio sin acoplarse a la interfaz ni a la tecnología de base de datos.

Tecnologías: TypeScript (tipado estricto de entidades).

Patron: Modelo del MVC.

Acoplamiento: Depende de: persistencia. No depende de vista ni controlador.

Componentes:

- Usuario, Perfil, Contacto: Gestion de identidad y roles (Consumidor, Vendedor, Tutor).
- Publicacion, Etiqueta, Imagen_Publicacion: Nucleo del intercambio de materiales y servicios.
- Resena: Sistema de calificaciones entre perfiles.
- Conversacion, Mensaje: Modelo de datos para la mensajería.
- TiempoDisponible, Tutoria: Gestion de sesiones académicas y disponibilidad.
- Acuerdo, Transaccion: Historial de intercambios y coordinación de entregas.

- Reporte, PalabrasRestringidas, Notificacion, Certificacion: Moderacion y credenciales.

Paquete: persistencia

Responsabilidad: Configura y expone la conexión a PostgreSQL mediante Prisma ORM. Actúa como puente entre el modelo y la base de datos, sin contener lógica de consultas directamente; esa responsabilidad es del paquete repository.

Tecnologías: Prisma ORM, PostgreSQL.

Patron: Delega las consultas al paquete repository. No implementa el patron directamente; lo habilita proveyendo el cliente de base de datos.

Acoplamiento: Depende de: infraestructura. Es usada por: modelo. Delega consultas a: repository.

Paquete: repository

Responsabilidad: Implementa el patron Repository. Cada clase encapsula todas las consultas de su dominio y expone una interfaz limpia al modelo. El modelo nunca conoce SQL ni detalles del esquema de base de datos; solo llama métodos como buscarPorId() o guardar().

Tecnologías: Prisma ORM.

Patron: Repository: separa la lógica de acceso a datos del dominio del negocio. Facilita pruebas unitarias al poder sustituir los repositorios por mocks.

Acoplamiento: Depende de: persistencia (cliente Prisma), infraestructura (ConectorPostgreSQL). Es usada por: modelo via persistencia.

Componentes:

- RepositorioUsuario: CRUD para Usuario, Perfil y Contacto.
- RepositorioPublicacion: CRUD para Publicacion, Imagen_Publicacion, Etiqueta y tablas de union.
- RepositorioMensaje: CRUD para Conversacion y Mensaje.
- RepositorioTutoria: CRUD para TiempoDisponible, Tutoria y Acuerdo.
- RepositorioReporte: CRUD para Reporte, Transaccion y PalabrasRestringidas.

Paquete: infraestructura

Responsabilidad: Capa más baja del sistema. Gestiona las conexiones a servicios externos y la configuración del entorno de despliegue. No contiene lógica de negocio; provee conexiones y utilidades a los demás paquetes.

Tecnologías: PostgreSQL (base de datos relacional principal), Redis (cache y pub/sub para WebSockets), Cloudinary (almacenamiento de imágenes en la nube), Docker y Docker Compose (contenedores y orquestación), Git y GitHub (control de versiones).

Patrón: No aplica un patrón específico; actúa como capa de integración con servicios de terceros.

Acoplamiento: No depende de ningún otro paquete del sistema. Es usada por: persistencia, tiempo_real, api_rest.

Componentes:

- ConectorPostgreSQL: Configura y expone el cliente de Prisma hacia la base de datos PostgreSQL.
- ConectorRedis: Instancia el cliente de Redis para cache de sesiones y pub/sub de WebSockets.
- ServicioCloudinary: Provee métodos para subir, actualizar y eliminar imágenes en Cloudinary.
- ConfiguracionDocker: Define los servicios (app, base de datos, Redis) en docker-compose.yml.
- GestorVariablesEntorno: Lee y valida las variables de entorno (.env) al iniciar el servidor.

Paquete: api_rest

Responsabilidad: Define y expone los endpoints HTTP del sistema bajo la arquitectura REST. Actúa como puerta de entrada de todas las peticiones del cliente al backend, enrutando cada solicitud al controlador correspondiente.

Tecnologías: Express.js, arquitectura REST sobre Cliente-Servidor.

Patrón: Front Controller: un único punto de entrada (servidor Express) que delega en routers especializados.

Acoplamiento: Depende de: controlador (para delegar lógica), infraestructura (para el despliegue). Es llamada por: vista.

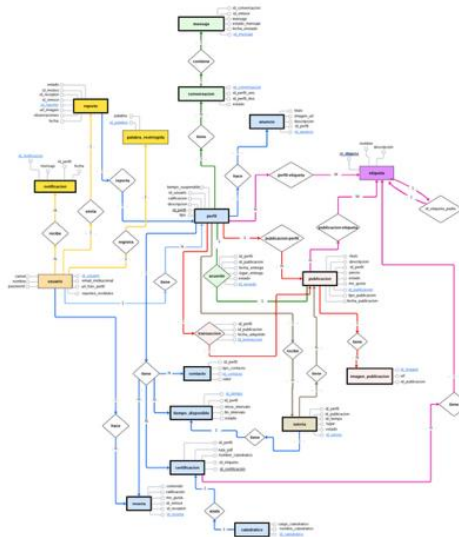
Componentes:

- RouterUsuario: Endpoints de registro, login, perfil y gestión de cuenta.
- RouterPublicacion: Endpoints de creación, edición, búsqueda y eliminación de publicaciones.
- RouterTutoria: Endpoints de disponibilidad, agendamiento y confirmación de tutorías.
- RouterModerador: Endpoints exclusivos del administrador para reportes y gestión de usuarios.

Persistencia de datos

Diagrama de Entidad-Relación

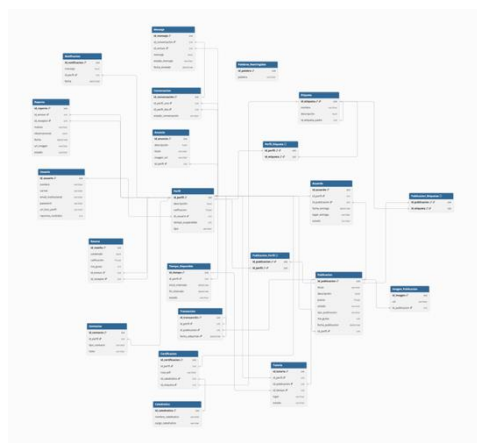
En este diagrama se visualiza la relación entre todas las entidades definidas para la base de datos.



En el siguiente [enlace](#) se encuentra a detalle el Diagrama de entidad-relación. Representa la estructura de la base de datos mostrando las principales entidades del sistema como los usuarios, perfil, publicaciones, chat, tutorías, así como sus atributos y relaciones entre ellos. Permite ver las interacciones mediante conversaciones, transacciones, reputación y moderación como reseñas y reportes.

Diagrama de Clases

En este diagrama se visualiza como será la implementación de la base de datos al DDL.



En el siguiente [enlace](#) se encuentra a detalle el Diagrama de clases. En este se muestran todas las entidades que serán usadas dentro de la base de datos junto a sus

atributos y las relaciones que tienen con otras entidades. A diferencia del DER, este es más cercano a la estructura dentro de la base de datos.

Diseño

Arquitectura

Cliente Servidor

Descripción

Es un modelo en el que las apps se dividen en 2 partes: El cliente que sería la interfaz que utiliza el usuario y el servidor, que está encargado de procesar solicitudes, manejar lógica del sistema y acceder a Base de Datos.

Ventajas

- Facilita mantenimiento y actualización del sistema
- Permite centralizar la lógica del sistema en el servidor
- Permite controlar seguridad y acceso de los datos
- Escala mejor cuando existen muchos usuarios

Desventajas

- Puede generar mucha carga en servidor si hay muchas solicitudes
- Requiere de una conexión a red para funcionar
- Depende de la disponibilidad del servidor

Razón por la cual se eligió

Se eligió porque se necesita que mantenga la información segura y centralizada cuando múltiples estudiantes interactúen con el sistema central.

API REST

Descripción

Es un estilo de arquitectura para diseñar sitios web que permite la comunicación entre el cliente y el servidor mediante las peticiones HTTP(GET, POST, etc.). El servidor expone recursos que el cliente puede consultar o modificar.

Ventajas

- Permite separar frontend y backend.

- Facilita la integración con otras aplicaciones.
- Utiliza estándares de internet como HTTP y JSON.

Desventajas

- No es ideal para comunicación en tiempo real.
- Puede requerir muchas solicitudes HTTP para ciertas operaciones.

Razón por la cual se eligió

Se eligió una API REST porque da más facilidad a la comunicación entre el frontend en Next.js y el backend. Manejando operaciones como crea, gestionar publicaciones, buscar materiales y enviar mensajes.

En Tiempo Real (WebSockets)

Descripción

Esta arquitectura usa tecnologías como WebSockets que permite una comunicación continua entre el cliente y el servidor, esto sin la necesidad de realizar múltiples solicitudes HTTP. Esto hace que el servidor pueda enviar información inmediatamente cuando ocurre un evento.

Ventajas

- Permite comunicación instantánea
- Reduce la necesidad de múltiples solicitudes HTTP
- Ideal para chats, notificaciones y actualizaciones en vivo

Desventajas

- Puede ser más complejo de implementar
- Requiere mayor gestión de conexiones activas

Razón por la cual se eligió

Se eligió esta arquitectura para implementar funcionalidades como mensajería entre los usuarios. Esto permitiendo que los mensajes, se envíen y reciban inmediatamente dentro del sitio.

Patrón de diseño

MVC (Model-View-Controller)

Descripción

Este patrón divide la aplicación en 3 componentes principales

- Vista: Representa la interfaz que es vista por el usuario.
- Modelo: Gestiona los datos y lógica
- Controlador: Maneja la comunicación entre la vista y el modelo.

Ventajas

- Permite organizar mejor el código.
- Facilita el mantenimiento del sistema.
- Permite trabajar en equipo separando responsabilidades.

Desventajas

- Puede ser más complejo de implementar en proyectos pequeños.
- Requiere una correcta estructura para evitar dependencias innecesarias.

Razón por la cual se eligió

Se eligió el patrón MVC porque permite separar claramente la interfaz de la lógica de negocio y el manejo de todos estos datos. Esto facilita el mantenimiento del sitio web al igual que el desarrolló.

Repository

Descripción

Este patrón se utiliza para abstraer el acceso de la base de datos, lo que crea una capa intermedia que maneja las operaciones de almacenamiento y recuperación de datos.

Ventajas

- Mejora la organización del código.
- Permite cambiar la base de datos sin afectar el resto del sistema.
- Facilita las pruebas del sistema.

Desventajas

- Agrega una capa adicional de complejidad.
- Puede requerir más código inicial.

Razón por la cual se eligió

Se eligió este patrón de diseño para poder mantener separado el acceso de la base de datos de la lógica del sistema. Esto facilita el manejo de información como usuarios, mensajes y productos.

Singleton

Descripción

Este patrón garantiza que una clase, tenga una única instancia en toda la ejecución de la aplicación. Esto permite que los componentes usen la misma instancia.

Ventajas

- Controla el acceso a recursos compartidos.
- Reduce el uso innecesario de memoria.
- Facilita el manejo de configuraciones globales.

Desventajas

- Puede generar dependencias globales.
- Puede dificultar las pruebas si se usa incorrectamente.

Razón por la cual se eligió

Se eligió este patrón para poder manejar elementos globales como conexiones a la base de datos asegurando que solamente exista una instancia activa.

Observer

Descripción

Este patrón permite que un objeto le notifique al resto de los objetos en caso de que cambie su estado. Se utiliza para sistemas de eventos o notificaciones.

Ventajas

- Permite comunicación flexible entre componentes.
- Facilita sistemas basados en eventos.
- Reduce el acoplamiento entre módulos.

Desventajas

- Puede generar muchas notificaciones si no se controla correctamente.
- Puede ser más difícil de depurar.

Razón por la cual se eligió

Se eligió para poder mantener la funcionalidad de notificar a los usuarios de mensajes o cambios en estado de productos, esto permite que el sistema actualice la información automáticamente.

Front-end

Next.js

Descripción

Es un framework de React que permite crear aplicaciones web con características como el renderizado del lado del servidor, generación de páginas estáticas y enrutamiento automático.

Ventajas

- Mejora el rendimiento y la carga de páginas.
- Permite SEO optimizado.
- Facilita la organización del proyecto.

Desventajas

- Puede tener una curva de aprendizaje mayor que React básico.
- Algunas configuraciones avanzadas pueden ser complejas.

Razón por la cual se eligió

Se eligió porque este permite desarrollar una app web rápida, escalable y bien estructurada para la plataforma.

TypeScript

Descripción

Esta es una extensión de JavaScript para agregar tipado estático, lo que permite detectar errores durante el desarrollo antes de ejecutar el código.

Ventajas

- Reduce errores en el código.
- Facilita el mantenimiento en proyectos grandes.
- Mejora la documentación del código.

Desventajas

- Requiere escribir más código.
- Puede tener una curva de aprendizaje inicial.

Razón por la cual se eligió

Esta se eligió para mejorar la seguridad del código y facilitar el desarrollo colaborativo.

CSS con metodología BEM

Descripción

Es una metodología para organización de las clases CSS de forma estructurada y reutilizable. Las siglas significan Block, Element, Modifier.

Ventajas

- Mantiene el CSS ordenado y escalable.
- Evita conflictos entre estilos.
- Facilita el mantenimiento.

Desventajas

- Puede generar nombres de clases más largos.
- Requiere disciplina para mantener la estructura.

Razón por la cual se eligió

Se eligió esta para mantener una estructura bien clara de los estilos del frontend, especialmente en una aplicación con muchas vistas.

Socket.io Client

Descripción

Esta es una biblioteca que permite al frontend establecer conexiones Websocket con el servidor para comunicación en tiempo real.

Ventajas

- Permite mensajes instantáneos.
- Maneja reconexiones automáticas.
- Compatible con múltiples navegadores.

Desventajas

- Requiere manejo adicional de eventos.
- Puede consumir más recursos si hay muchas conexiones.

Razón por la cual se eligió

Se eligió porque de esta manera se puede implementar correctamente la mensajería en tiempo real entre usuarios.

Zustand

Descripción

Es una biblioteca para gestionar el estado global de una aplicación React de forma simple y eficiente.

Ventajas

- Es ligero y fácil de usar
- Reduce la complejidad del manejo de estados
- No requiere muchas configuraciones

Desventajas

- Tiene menos herramientas que soluciones más grandes como Redux
- Comunidad más pequeña

Razón por la cual se eligió

Se eligió para manejar estados globales como usuario autenticado, mensajes o los productos seleccionados.

React Hook Form

Descripción

Es una biblioteca para manejar formularios en aplicaciones React. Esto permitiendo validar y controlar los datos ingresados por el usuario.

Ventajas

- Alto rendimiento
- Reduce la cantidad de código necesario
- Validaciones simples

Desventajas

- Puede requerir aprendizaje inicial
- Algunas validaciones complejas pueden requerir configuración adicional

Razón por la cual se eligió

Se eligió para poder manejar formularios como registro de usuario, creación de publicación de producto y solicitud de tutorías.

Date-fns

Descripción

Es una biblioteca de Javascript que permite manipular y formatear fechas de forma sencilla.

Ventajas

- Ligera y modular
- Fácil de usar
- Buen rendimiento

Desventajas

- No incluye tantas funcionalidades como otras bibliotecas más grandes
- Requiere importar funciones específicas

Razón por la cual se eligió

Se eligió para poder manejar fechas y horarios de tutorías, publicaciones y mensajes dentro del sitio web.

Back-end

Node.js

Descripción

Entorno de ejecución para JavaScript.

Ventajas

- Alta eficiencia en aplicaciones con múltiples conexiones simultáneas.
- Permite utilizar JavaScript tanto en el frontend como en el backend.
- Acceso a la mayor biblioteca de paquetes de software en el mundo.

Desventajas

- Rendimiento deficiente en tareas de procesamiento pesado.
- Algunos cambios entre versiones de Node pueden romper la compatibilidad de módulos antiguos.

Razón por la cual se eligió

Se eligió Node.js como entorno de ejecución para el backend por su capacidad para manejar múltiples solicitudes de manera eficiente mediante un modelo de ejecución

asincrónico. Esto es especialmente útil para aplicaciones web con alta interacción entre usuarios, como el sistema propuesto que incluye chat, consultas de publicaciones y gestión de tutorías. Además, Node.js cuenta con un ecosistema amplio de librerías que facilitan el desarrollo de aplicaciones modernas en el servidor.

Express.js

Descripción

Framework web para Node.js que proporciona una capa de abstracción sobre los módulos HTTP nativos, facilitando el enrutamiento y la gestión de middleware.

Ventajas

- No impone una estructura rígida, permitiendo total libertad al desarrollador.
- Facilidad para interceptar y procesar peticiones HTTP en cualquier etapa.
- Curva de aprendizaje muy baja para quienes ya dominan JavaScript y Node.

Desventajas

- En proyectos grandes, la organización del código puede volverse caótica sin estándares manuales.
- Requiere instalar muchos paquetes externos para tareas comunes.
- El desarrollador es responsable de implementar la mayoría de las protecciones contra ataques comunes.

Razón por la cual se eligió

Se eligió Express.js porque es un framework ligero y ampliamente utilizado para construir APIs en Node.js. Permite organizar fácilmente las rutas del sistema, manejar solicitudes HTTP y estructurar el backend de forma clara, facilitando el desarrollo de los servicios que utilizará el frontend para interactuar con el sistema.

TypeScript

Descripción

Superset de JavaScript que añade tipado estático opcional y características de programación orientada a objetos.

Ventajas

- Detecta errores de tipo durante el desarrollo, antes de ejecutar el código.
- Permite una refactorización sencilla

Desventajas

- Requiere configurar un compilador (tsconfig) y herramientas de construcción adicionales.
- Añade un paso extra que puede ralentizar el flujo de trabajo en proyectos masivos.
- Obliga a escribir más código para definir interfaces, tipos y genéricos.

Razón por la cual se eligió

Se eligió TypeScript porque permite agregar tipado estático al código JavaScript, lo que ayuda a detectar errores durante el desarrollo y mejora la mantenibilidad del sistema. Además, facilita la colaboración entre los miembros del equipo al definir claramente las estructuras de datos utilizadas en el backend.

Socket.io

Descripción

Librería que permite la comunicación de baja latencia y basada en eventos entre clientes y servidores, utilizando WebSockets como transporte principal.

Ventajas

- Comunicación bidireccional instantánea con latencia mínima.
- Gestiona automáticamente la recuperación del enlace si se pierde la conexión.

Desventajas

- Más pesado que usar WebSockets puros debido a sus características adicionales.
- Requiere usar la librería específica de Socket.io en el frontend y no sirve un cliente WebSocket estándar.
- Mantener miles de conexiones abiertas consume mucha memoria RAM en el servidor.

Razón por la cual se eligió

Se eligió Socket.io para implementar comunicación en tiempo real dentro del sistema. Esta tecnología permite desarrollar funcionalidades como el chat entre estudiantes y tutores, notificaciones de mensajes y actualizaciones inmediatas sin necesidad de recargar la página.

Zod

Descripción

Biblioteca de declaración y validación de esquemas con inferencia de tipos de TypeScript.

Ventajas

- Crea automáticamente tipos de TypeScript a partir de los esquemas de validación.
- Valida que los datos externos (APIs, formularios) sean realmente lo que el código espera

Desventajas

- Añade peso extra a la aplicación si se utiliza intensivamente en el frontend.
- La validación de objetos masivos y anidados en cada petición puede generar un pequeño impacto de latencia.

Razón por la cual se eligió

Se eligió Zod para validar los datos que llegan al servidor desde el cliente. Esto permite asegurar que la información recibida tenga el formato correcto antes de ser procesada o almacenada en la base de datos, lo cual ayuda a prevenir errores y mejorar la seguridad del sistema.

JSON Web Token (JWT)

Descripción

Estándar para definir un modo compacto de transmitir información como un objeto JSON de forma segura.

Ventajas

- Un solo token puede funcionar en múltiples microservicios si comparten la misma clave secreta.
- Fácil de enviar a través de encabezados HTTP o URLs.

Desventajas

- Al contener datos, los tokens pueden crecer y aumentar el peso de cada petición HTTP.

Razón por la cual se eligió

Se eligió JSON Web Token para implementar el sistema de autenticación de usuarios. Esta tecnología permite verificar la identidad de los usuarios mediante tokens firmados que pueden enviarse en cada solicitud al servidor, facilitando el manejo de sesiones de manera segura y eficiente.

Bcrypt

Descripción

Biblioteca para el hashing seguro de contraseñas

Ventajas

- Permite aumentar la dificultad del hash conforme el hardware se vuelve más potente.
- Algoritmo ampliamente probado y considerado seguro para almacenamiento de credenciales.

Desventajas

- El proceso de hashing es intensivo y puede ralentizar el servidor bajo ráfagas de logins.
- Solo sirve para contraseñas; no es un algoritmo de cifrado para recuperar datos originales.

Razón por la cual se eligió

Se eligió bcrypt para proteger las contraseñas de los usuarios mediante un proceso de cifrado antes de almacenarlas en la base de datos. Esto permite mejorar la seguridad del sistema y reducir el riesgo de credenciales en caso de acceso no autorizado a la base de datos.

Prisma ORM

Descripción

ORM (Object-Relational Mapper) que traduce esquemas de base de datos a tipos de TypeScript.

Ventajas

- Type-safety total

- Gestiona los cambios en el esquema de la base de datos de forma declarativa.

Desventajas

- Abstracción pesada
- Algunas operaciones SQL muy específicas o reportes complejos son difíciles de expresar sin usar SQL puro.

Razón por la cual se eligió

Se eligió Prisma ORM porque permite interactuar con la base de datos de forma estructurada mediante código en lugar de escribir consultas SQL manualmente. Además, se integra fácilmente con TypeScript, facilita la gestión de migraciones del esquema de la base de datos y mejora la organización del acceso a los datos dentro del backend.

Base de dato y almacenamiento

PostgreSQL

Descripción

PostgreSQL es un sistema de gestión de bases de datos relacional de código abierto que permite almacenar y gestionar datos estructurados mediante tablas relacionadas. Utiliza el lenguaje SQL para realizar consultas y operaciones sobre los datos.

Ventajas

- Permite manejar relaciones complejas entre entidades mediante claves foráneas.
- Garantiza integridad y consistencia de los datos mediante transacciones.
- Es altamente confiable y escalable para aplicaciones web.
- Cuenta con una gran comunidad y documentación.

Desventajas

- Puede requerir mayor configuración inicial que otras bases de datos más simples.
- No es tan flexible como las bases de datos no relacionales para datos no estructurados.

Razón por la cual se eligió

Se eligió PostgreSQL porque el sistema requiere manejar múltiples relaciones entre entidades como usuarios, publicaciones, tutorías, mensajes y reseñas. Una base de

datos relacional permite estructurar estos datos de forma organizada y garantizar la integridad de la información.

Redis

Descripción

Redis es una base de datos en memoria que se utiliza principalmente como sistema de caché o almacenamiento temporal de datos. Permite acceder a la información de manera muy rápida debido a que los datos se almacenan en memoria.

Ventajas

- Acceso extremadamente rápido a los datos.
- Mejora el rendimiento del sistema al reducir consultas a la base de datos principal.
- Permite manejar información temporal como sesiones o tokens.

Desventajas

- Los datos almacenados en memoria pueden perderse si el sistema se reinicia (si no se configura persistencia).
- No está diseñado para almacenar grandes volúmenes de información permanente.

Razón por la cual se eligió

Se eligió Redis para manejar datos temporales y mejorar el rendimiento del sistema. En este proyecto puede utilizarse para almacenar información como tokens de autenticación bloqueados, sesiones activas o caché de consultas frecuentes.

Cloudinary

Descripción

Cloudinary es un servicio de almacenamiento en la nube especializado en archivos multimedia como imágenes y videos. Permite almacenar, optimizar y servir archivos de manera eficiente mediante una red de distribución de contenido (CDN).

Ventajas

- Reduce el uso de almacenamiento en el servidor principal.
- Optimiza automáticamente las imágenes para mejorar su rendimiento.
- Permite entregar archivos multimedia de forma rápida mediante CDN.

Desventajas

- Depende de un servicio externo para almacenar los archivos multimedia.
- Puede tener límites de uso en planes gratuitos.

Razón por la cual se eligió

Se eligió Cloudinary para gestionar el almacenamiento de imágenes de publicaciones y perfiles de usuario. Esto permite evitar que el servidor principal almacene archivos multimedia y mejora la velocidad de carga de las imágenes en la aplicación.

Otros

Docker

Descripción

Docker es una plataforma de contenerización que permite empaquetar aplicaciones junto con todas sus dependencias dentro de contenedores portables.

Ventajas

- Permite ejecutar el sistema en cualquier entorno sin problemas de configuración.
- Facilita la distribución y despliegue del sistema.
- Garantiza que todos los miembros del equipo utilicen el mismo entorno de desarrollo.

Desventajas

- Puede requerir aprendizaje adicional para configurar correctamente los contenedores.
- Puede aumentar ligeramente el consumo de recursos del sistema.

Razón por la cual se eligió

Se eligió Docker para garantizar que el sistema pueda ejecutarse de manera consistente en diferentes entornos de desarrollo y producción, facilitando el despliegue del proyecto en un servidor propio.

Docker Compose

Descripción

Docker Compose es una herramienta que permite definir y ejecutar aplicaciones compuestas por múltiples contenedores mediante un archivo de configuración.

Ventajas

- Permite ejecutar múltiples servicios con un solo comando.
- Facilita la gestión de aplicaciones que requieren varios contenedores.
- Simplifica la configuración del entorno de desarrollo.

Desventajas

- Puede aumentar la complejidad del archivo de configuración si se agregan muchos servicios.
- Requiere conocer la estructura de los contenedores del sistema.

Razón por la cual se eligió

Se eligió Docker Compose para gestionar los diferentes servicios del sistema, como el frontend, backend, base de datos y Redis, permitiendo ejecutar toda la aplicación de forma organizada y centralizada.

Git

Descripción

Git es un sistema de control de versiones distribuido que permite registrar los cambios realizados en el código de un proyecto.

Ventajas

- Permite mantener un historial completo de cambios en el código.
- Facilita el trabajo colaborativo entre varios desarrolladores.
- Permite revertir cambios en caso de errores.

Desventajas

- Puede ser complejo para usuarios sin experiencia previa.
- Requiere seguir buenas prácticas para evitar conflictos en el código.

Razón por la cual se eligió

Se eligió Git para gestionar el control de versiones del proyecto, permitiendo a los miembros del equipo colaborar en el desarrollo del sistema y mantener un historial organizado de cambios.

GitHub

Descripción

GitHub es una plataforma que permite alojar repositorios de código utilizando Git y facilita la colaboración en proyectos de desarrollo de software.

Ventajas

- Permite almacenar el código del proyecto de forma centralizada.
- Facilita la colaboración entre desarrolladores mediante ramas y pull requests.
- Permite mantener un historial de cambios accesible en línea.

Desventajas

- Dependencia de una plataforma externa para almacenar el código.
- Puede requerir conexión a internet para acceder a los repositorios.

Razón por la cual se eligió

Se eligió GitHub para alojar el repositorio del proyecto y permitir que todos los miembros del equipo puedan colaborar en el desarrollo del sistema mediante control de versiones.

Planificación de tiempo

LOGT

Josue Hernández González - 24770

Fecha	Inicio	Fin	Tiempo interrupción	Delta tiempo	Fase	Comentarios
09/03	18:00	18:30	5 min	25 min	Prototipo iteración 1	Entrevista a usuarios perfil tutor
10/03	2:00	2:25	0 min	25 min	Resumen	Basado en corte 1 y 2
11/03	18:00	18:30	5 min	25 min	Prototipo iteración 2	Entrevista a usuarios perfil vendedor
12/03	8:00	8:30	0 min	30 min	Requisitos funcionales	Requisitos funcionales del 17 al 22
12/03	10:00	10:30	0 min	30 min	Prototipo iteración 3	Figma vista lista de usuarios y lista de reportes
14/03	20:00	21:00	10 min	50 min	Diseño base datos	Corrección y reestructuración de entidades y atributos final
14/03	22:00	24:00	10 min	110 min	Prototipo iteración 3	Revisión completa de todas las ventanas y vistas en figma
15/03	17:00	18:30	15 min	75 min	Ajustes de repartición de trabajo	Reunión del grupo, revisión y busca de errores, asignar partes de acuerdo a feedback del grupo.
16/03	12:20	1:10	5 min	45 min	Cambio de requisitos funcionales	Lista de cambios de requisitos funcionales tras las iteraciones de usuarios.
16/03	1:30	3:00	10 min	80 min	Tecnologías	Uso de tecnologías para base de datos, almacenamiento, dockerización y gestor de versiones.
16/03	7:00	9:30	0 min	150 min	Revisión completa	Revisión completa de documentación y asignar correcciones.
16/03	11:00	11:30	5 min	25 min	Tecnologías	Agregar el porqué se usarán ciertas tecnologías.

Jackelyn Nicolle Girón Villacinda – 24737

Fecha	Inicio	Fin	Tiempo interrupción	Delta tiempo	Fase	Comentarios
09/03	18:25	18:40	0 min	15 min	Entrevistas	Entrevista a usuario vendedor
09/03	18:45	19:00	0 min	15 min	Prototipo	Cambios en prototipo
11/03	18:00	18:20	0 min	20 min	Entrevistas	Entrevista usuario vendedor
11/03	16:30	19:20	40 min	130 min	Prototipo	Creación de proyecto en figma, estilos y Vista perfil tutor
12/03	15:00	15:45	0 min	45 min	Requisitos Funcionales	Requisitos funcionales del 1 al 5
12/03	17:00	20:20	15 min	185 min	Prototipo	Vista Login, Vista Register y Pagina principal
14/03	22:00	24:00	10 min	110 min	Prototipo iteración 3	Revisión completa de todas las ventanas y vistas en figma
15/03	18:05	23:15	20 min	310 min	Prototipo final	Creación de todas las vistas faltantes en figma y cambios
16/03	0:15	0:50	0 min	35 min	Documento	Listado de iteraciones y cambios
16/03	11:20	12:00	20 min	20 min	Documento	Introducción
16/03	22:00	22:25	0 min	25 min	Bitácora	Se incluye la información de las entrevistas en la bitacora

Alejandro Manuel Jerez Melgar – 24678

Fecha	Inicio	Fin	Tiempo interrupción	Delta tiempo	Fase	Comentarios
08/03	9:30	12:00	0 min	210 min	Análisis, persistencia de datos	Primera versión de la BD, declaración de

						entidades, atributos, PK, relaciones
08/03	13:00	22:00	60 min	480 min	Análisis, persistencia de datos	Primera versión de la BD, proceso de normalización y creación de diagramas
09/03	08:00	09:30	0 min	90 min	Análisis, persistencia de datos	Primera versión de la BD, creación del DDL
09/03	18:00	18:30	5 min	25 min	Prototipo iteración 1	Entrevista a usuarios perfil tutor
11/03	18:00	18:30	5 min	25 min	Prototipo iteración 2	Entrevista a usuarios perfil tutor
12/03	8:00	9:00	0 min	60 min	Requisitos funcionales	Requisitos funcionales del 11 al 16
12/03	10:00	12:00	0 min	120 min	Prototipo iteración 3	Figma de la interfaz de tutorías, filtro tutorías, crear tutoría y crear producto
12/03	14:00	17:00	30 min	150 min	Prototipo iteración 3	Figma de la interfaz de tutorías, filtro tutorías, crear tutoría y crear producto
12/03	18:00	19:25	0 min	85 min	Prototipo iteración 3	Figma de la interfaz de tutorías, filtro tutorías, crear tutoría y crear producto
15/03	10:00	13:00	30 min	150 min	Diagrama de Clases y paquetes	Diagrama y definición de clases y paquetes

Juan Daniel Orodoñez Sierra – 24979

Fecha	Inicio	Fin	Tiempo interrupción	Delta tiempo	Fase	Comentarios
09/03	18:00	18:30	5 min	25 min	Prototipo iteración 1	Entrevista a usuarios perfil tutor

11/03	18:00	18:30	5 min	25 min	Prototipo iteración 2	Entrevista a usuarios perfil tutor
11/03	16:00	17:00	0 min	60 min	Requisitos funcionales	Requisitos funcionales del 6 al 10
12/03	10:00	12:00	20 min	180 min	Prototipo iteración 3	Figma de la interfaz de perfil vendedor, vista emergente de reportes y vista de negocios.
15/03	21:00	00:30	20 min	210 min	Tecnologías	Investigación sobre tecnologías de Frontend, arquitecturas y patrones de diseño.
16/03	00:50	1: 30	5 min	35 min	Bitácora de Entrevistas	Anotaciones sobre la evidencia de las entrevistas realizadas.
16/03	22:00	23:00	10 min	60 min	Revisión Final	Revisión del documento comparando comentarios de entregas pasadas.

Oscar Rodrigo Rompich Cotzoyay – 24880

Fecha	Inicio	Fin	Tiempo interrupción	Delta tiempo	Fase	Comentarios
09/03	18:00	18:30	5 min	25 min	Prototipo iteración 1	Entrevista a usuarios perfil tutor
11/03	18:00	18:30	5 min	25 min	Prototipo iteración 2	Entrevista a usuarios perfil tutor
12/03	8:00	9:00	0 min	60 min	Requisitos funcionales	Requisitos funcionales del 22 al 28
12/03	10:00	12:00	20 min	180 min	Prototipo iteración 3	Figma de la interfaz de materiales, filtros, detalles de publicaciones, y perfil de tutor.
14/04	12:00	18:00	20 min	360 min	Base de Datos	Diagrama de entidad relación

15/03	19:00	24:00	20 min	240 min	Base de Datos	Diagrama del modelo relacional
16/03	1:00	1:30	0 min	30 min	Bitácora de Entrevistas	Anotaciones sobre la evidencia de las entrevistas realizadas.
17/03	00:00	1:30	0 min	90 min	Tecnologías	Investigación sobre tecnologías de backend.

Sergio Estuardo Tan Coromac - 24759

Fecha	Inicio	Fin	Tiempo interrupción	Delta tiempo	Fase	Comentarios
09/03	18:25	18:40	0 min	15 min	Prototipo	Entrevista a usuario vendedor
11/03	18:00	18:30	0 min	15 min	Prototipo	Entrevista a usuario vendedor
11/03	19:00	19:30	0 min	30 min	Requisitos funcionales	Requisitos funcionales del 29 al 34
11/03	22:30	23:59	10 min	79 min	Prototipo iteración 3	Figma vista de detalles de tutorías
12/03	0:00	1:30	5 min	85 min	Prototipo iteración 3	Figma vista de chat entre usuarios
15/03	23:00	23:59	10 min	49 min	Presentación de corte	Creación y avances de la presentación del corte
16/03	8:30	9:00	0 min	30 min	Bitácora	Refinamiento a anotaciones en las bitácoras
15/03	9:00	10:30	25 min	75 min	Presentación de corte	Avances de la presentación del corte

Conclusiones y reflexión de gestión de tiempo

Se logró finalizar exitosamente la retroalimentación de los usuarios, se diseñó la base de datos con colaboración de todos los participantes y se planeó el uso de las tecnologías para el proyecto.